Vel Tech Group of Institutions

Name: UMA MAHESHWARI S
Email: vtu28408@veltech.edu.in
Roll no: vtu28408
Phone: null
Branch: VTU
Department: CSE with AIDS
Batch: 2028
Degree: B.Tech - Computer Science and Engineering with AIDS

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 7_COD

Attempt : 1
Total Mark : 30
Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Prim’s Algorithm for Minimum Spanning Tree (MST):

Sita is planning to lay roads to connect 5 locations in her city. The distances between locations are given as an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to 4).

Help her find the Minimum Spanning Tree (MST) using Prim's Algorithm so that all locations are connected with the minimum total road length. Print each MST edge and its weight in the order they are added to the MST.

Input Format

The input consists of 5 lines, each containing 5 space-separated integers,

representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two locations.

Output Format

The first line of output prints the header: 'Edge \tWeight'

Each of the next V-1 lines prints one MST edge in the format:

'u - v\tw'

where u is the parent location, v is the child location, and w is the integer edge weight, printed in the order edges are added to the MST (MST discovery order).

Refer to the sample output for formatting specifications.

Refer to the sample output for format specifications.

Sample Test Case

Input: 0 2 0 6 0
2 0 3 8 5
0 3 0 8 7
6 8 0 0 9
0 5 7 9 0
Output: Edge Weight
0 - 1 2
1 - 2 3
1 - 4 5
0 - 3 6

Answer

```
#include <stdio.h>
#define V 5
#define INF 999999

int minKey(int key[], int mstSet[]) {
    int min = INF, min_index;
```

```
    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

int main() {
    int graph[V][V];

    // Input adjacency matrix
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int parent[V];     // Store MST
    int key[V];        // Minimum weights
    int mstSet[V];     // Included in MST or not

    // Initialize
    for (int i = 0; i < V; i++) {
        key[i] = INF;
        mstSet[i] = 0;
    }

    key[0] = 0;      // Start from vertex 0
    parent[0] = -1;  // Root

    // Build MST
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);
        mstSet[u] = 1;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
```

```
        }
    }
}

    // Output
    printf("Edge \tWeight ");
    for (int i = 1; i < V; i++) {
        printf("%d - %d\t%d ", parent[i], i, graph[parent[i]][i]);
    }

    return 0;
}// You are using GCC
```

Status : Wrong **Marks : 0/10**

2. Problem statement

Alex is a network engineer tasked with laying cables to connect multiple offices in the city. Each possible cable connection between two offices has an associated cost. To minimize the total cabling cost while ensuring all offices are connected, Alex decides to use Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the office network.

Given a connected, undirected, weighted graph with V vertices (numbered 0 to V-1) and E edges, find the MST and print each edge included along with the total minimum cost.

Input Format

The first line contains an integer V, representing the number of vertices (numbered 0 to V-1).

The second line contains an integer E, representing the number of edges.

Each of the next E lines contains three space-separated integers u, v, and w, representing an undirected edge between vertex u and vertex v with weight w

Output Format

The first line of output prints: 'Edges in the constructed MST:'

Each of the next V-1 lines prints one MST edge in the format:

'u -- v == weight'

where u and v are vertex numbers (integers) and weight is the edge cost (integer), printed in the order edges are added to the MST (ascending weight order).

The last line prints: 'Minimum Spanning Tree: totalCost'

where totalCost is an integer representing the total MST weight.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
3
0 1 2
1 2 2
0 2 2
Output: Edges in the constructed MST:
0 -- 1 == 2
1 -- 2 == 2
Minimum Spanning Tree: 4

Answer

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

int parent[105];

int find(int x) {
    if (parent[x] != x)
```

```
    parent[x] = find(parent[x]);
    return parent[x];
}

void unionSet(int a, int b) {
    int pa = find(a);
    int pb = find(b);
    parent[pa] = pb;
}

int cmp(const void *a, const void *b) {
    return ((Edge *)a)->w - ((Edge *)b)->w;
}

int main() {
    int V, E;
    scanf("%d %d", &V, &E);

    Edge edges[E];

    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    qsort(edges, E, sizeof(Edge), cmp);

    for (int i = 0; i < V; i++)
        parent[i] = i;

    int totalCost = 0;
    int count = 0;

    printf("Edges in the constructed MST:\n");

    for (int i = 0; i < E && count < V - 1; i++) {
        int u = edges[i].u;
        int v = edges[i].v;
        int w = edges[i].w;

        if (find(u) != find(v)) {
            unionSet(u, v);
            printf("%d -- %d == %d\n", u, v, w);
```

```
            totalCost += w;
            count++;
        }
    }

    printf("Minimum Spanning Tree: %d", totalCost);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

You are designing a data compression tool that uses Huffman Coding to encode data efficiently.

Write a program to read a set of characters along with their corresponding Huffman variable-length binary codes, and use them to generate the encoded binary string for a given input message.

It is guaranteed that every character in the input message has a corresponding Huffman code in the provided code table.

Input Format

The first line contains an integer N (integer), representing the number of characters in the code table.

Each of the next N lines contains a character C and its corresponding Huffman code (a binary string of 0s and 1s), separated by a space.

The last line contains the input message (a string of characters) that needs to be encoded. Every character in the message is guaranteed to have a corresponding entry in the code table.

Output Format

The output prints 'Coded Data: ' followed by a binary string (a sequence of 0s and 1s) representing the encoded message, obtained by concatenating the Huffman code of each character in the input message in order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10
D 0100
U 0000
F 0101
M 0001
A 0010
N 0011
C 1100
O 1101
E 1110
H 1111
HUFF

Output: Coded Data: 1111000001010101

Answer

```
#include <stdio.h>
#include <string.h>

int main() {
    int N;
    scanf("%d", &N);

    char codes[26][25]; // store codes for A-Z
    char ch, code[25];

    // Initialize empty
    for (int i = 0; i < 26; i++)
        codes[i][0] = '\0';

    // Read mapping
    for (int i = 0; i < N; i++) {
        scanf(" %c %s", &ch, code);
        strcpy(codes[ch - 'A'], code);
    }

    char message[105];
```

```
    scanf("%s", message);

    printf("Coded Data: ");

    // Encode message
    for (int i = 0; message[i] != '\0'; i++) {
        printf("%s", codes[message[i] - 'A']);
    }

    return 0;
}// You are using GCC
```

Status : Correct **Marks : 10/10**